

VULNERABILITY ASSESSMENT REPORT (VAR)

MAINCRAFTS TECHNOLOGY CYBERSECURITY INTERNSHIP PROGRAM

DOCUMENT METADATA

- **ASSIGNED TASK:** Task-3: Vulnerability Scanning & Assessment
- **PREPARED BY:** Harshil
- **DATE OF SUBMISSION:** July 20, 2026
- **TARGET SYSTEM:** OWASP Juice Shop Sandbox Environment (Staging)
- **SUBMITTED TO:** Internship Review & Approval Board, Maincrafts Technology
- **STATUS:** ● Pending Approval

TABLE OF CONTENTS

1. [Executive Summary](#)
2. [Scope of Assessment & Engagement Rules](#)
3. [Methodology & Tools Used](#)
4. [Step-by-Step Activities & Command Outputs](#)
5. [Detailed Technical Findings & Impact Analysis](#)
6. [Remediation & Action Plan](#)
7. [Conclusion & Next Steps](#)

1. EXECUTIVE SUMMARY

Maincrafts Technology commissioned a professional-grade vulnerability assessment of the sandboxed staging environment deployed during Task-2. This assessment was performed by Cybersecurity Analyst Intern **Harshil** on July 20, 2026. The target of this evaluation was an intentionally vulnerable web application instance (OWASP Juice Shop) hosted locally within an isolated virtualization boundary.

The primary objective of this exercise was to identify, classify, and document potential network-level and web application-level security vulnerabilities prior to any active exploitation or patching phases (which are reserved for Task-4).

HIGH-LEVEL FINDINGS SUMMARY

Severity	Count	Action Required
● Critical	0	No immediate emergency response needed
● High	1	Immediate mitigation recommended (Sensitive Directory Exposure)
● Medium	3	Address during next maintenance cycle (Missing Security Headers, HTTP Verb Abuse)
● Low	1	Informational baseline updates (Technology Stack Information Disclosure)

If left unmitigated in a production environment, these vulnerabilities could allow malicious actors to perform directory traversal, execute Clickjacking attacks, craft targeted exploits based on leaked technology stacks, or abuse administrative APIs. Immediate remediation via secure server configuration, input sanitization, and security header enforcement is highly recommended.

2. SCOPE OF ASSESSMENT & ENGAGEMENT RULES

This assessment was carried out under strict, predefined Rules of Engagement (RoE) to ensure safety, containment, and system integrity.

RULES OF ENGAGEMENT	
IN-SCOPE:	
✓ Local sandboxed lab systems only (Target IP Range: 192.168.56.0/24)	
✓ Passive reconnaissance & active vulnerability scanning	
✓ Network layer mapping and web directory fuzzing	
✓ Nessus, Nmap, Nikto, Dirsearch, and Burp Suite tools	
OUT-OF-SCOPE:	
✗ Public internet, production systems, third-party sites	
✗ Active exploitation, reverse shells, or payload execution	
✗ Denial of Service (DoS/DDoS) and system destruction	

3. METHODOLOGY & TOOLS USED

The assessment followed a structured, industry-standard lifecycle aligned with the **OWASP Testing Methodology** and **NIST SP 800-115**:

- 1. Information Gathering:** Performing low-noise discovery scans to verify live systems within the private host-only virtual network segment.
- 2. Scanning & Port Enumeration:** Mapping open network ports and interrogating services to discover exact versioning and technology headers.
- 3. Web Server Assessment:** Running dedicated web vulnerability scanners to detect missing configuration safety mechanisms and unsafe application structures.
- 4. Endpoint Discovery:** Leveraging directory brute-forcing (fuzzing) to discover hidden administration paths or misplaced files.
- 5. Reporting & Analysis:** Consolidating raw scanner logs into this formal, prioritized audit report with risk ratings (CVSS v3.1 standard) and realistic remediation steps.

AUTOMATED TOOL STACK

- **Nmap v7.94:** Used for live host discovery, port scanning, and service version enumeration.
- **Nikto v2.5.0:** Used for identifying outdated server software, configuration flaws, and missing HTTP headers.
- **Dirsearch v0.4.3:** Used for multi-threaded hidden directory and file discovery on the web target.

4. STEP-BY-STEP ACTIVITIES & COMMAND OUTPUTS

All scans were executed from the attacking instance running Kali Linux (hostname: harshil, IP: 192.168.56.101) targeting the web server machine (IP: 192.168.56.102).

Phase 1: Host Discovery & Network Reconnaissance

To confirm that the targeted web host was live, reachable, and correctly bound to the host-only adapter segment (vboxnet0), an active Nmap ping sweep was performed.

Command Executed:

```
harshil@harshil:~$ nmap -sn 192.168.56.0/24 -oN ~/lab/scans/host-discovery.txt
```

Terminal Output:

```
Starting Nmap 7.94 ( https://nmap.org ) at 2026-07-20 10:15 IST
Nmap scan report for 192.168.56.1
Host is up (0.00028s latency).
Nmap scan report for target-juice-shop (192.168.56.102)
Host is up (0.00045s latency).
MAC Address: 08:00:27:8D:BC:39 (Oracle VirtualBox virtual NIC)
Nmap done: 256 IP addresses (2 hosts up) scanned in 1.45 seconds
```

Assessment Note: Confirmed live target footprint at 192.168.56.102.

Phase 2: Detailed Port & Service Enumeration

An aggressive service scanning configuration was executed against the discovered host to map out open TCP ports, running services, and their operating software versions.

Command Executed:

```
harshil@harshil:~$ nmap -sV -p- 192.168.56.102 -oN ~/lab/scans/full-service-scan.txt
```

 VISUAL PROOF #1: Network Port Mapping

TERMINAL SCAN - PORT & SERVICE ENUMERATION				
harshil@harshil:~\$ nmap -sV -p- 192.168.56.102				
Nmap scan report for target-juice-shop (192.168.56.102)				
Host is up (0.00052s latency).				
Not shown: 65533 closed tcp ports (reset)				
PORT STATE SERVICE VERSION				
22/tcp	open	ssh	OpenSSH 8.9p1 Ubuntu 3ubuntu0.1 (Ubuntu Linux)	
3000/tcp	open	http	Node.js Express framework (OWASP Juice Shop)	
MAC Address: 08:00:27:8D:BC:39 (Oracle VirtualBox virtual NIC)				
Service detection performed.				
Nmap done: 1 IP address (1 host up) scanned in 12.34 seconds				

Phase 3: Web Server & Application Vulnerability Scanning

With port 3000 confirmed as running an HTTP service, a targeted Nikto scan was deployed to discover default system pages, dangerous HTTP verbs, and critical missing client-side security policies.

Command Executed:

```
harshil@harshil:~$ nikto -h http://192.168.56.102:3000 -o ~/lab/scans/nikto.txt
```

 VISUAL PROOF #2: Web Configuration Scan

NIKTO WEB VULNERABILITY SCAN	
- Nikto v2.5.0	

+ Target IP:	192.168.56.102
+ Target Port:	3000
+ Start Time:	2026-07-20 10:22:15 (GMT+5.5)

+ Server: No banner retrieved	
+ /: The anti-clickjacking X-Frame-Options header is not present.	
+ /: The X-Content-Type-Options header is not set.	
+ /: The Content-Security-Policy header is not present.	
+ /: Web server reveals tech stack: 'x-powered-by: Express'	
+ /: HTTP Methods Allowed: GET, HEAD, POST, PUT, DELETE, OPTIONS	
+ /: Dangerous HTTP methods (PUT, DELETE) are enabled.	

+ 7534 targets checked	

Phase 4: Hidden Directory & Endpoint Fuzzing

To systematically reveal administrative paths, directory indexes, or hidden structures, dirsearch was run against the application using a medium wordlist.

Command Executed:

```
harshil@harshil:~$ dirsearch -u http://192.168.56.102:3000 -e php,html,js,json,bak,txt -o ~/lab/scans/dirsearch
```

 VISUAL PROOF #3: Endpoint Fuzzing & Directory Listing

DIRSEARCH DIRECTORY FUZZING LOGS	
[10:30:11] Starting: dirsearch v0.4.3	
[10:30:12]	301 - 169B - /assets -> redirect to /assets/
[10:30:14]	200 - 2KB - /assets/index.html
[10:30:18]	200 - 980B - /api/Users
[10:30:22]	301 - 165B - /ftp -> redirect to /ftp/
[10:30:23]	200 - 5KB - /ftp/
[10:30:23]	200 - 450B - /ftp/coupons_2026.pdf.bak
[10:30:25]	200 - 1KB - /ftp/security_audit_stage1.json
[10:30:31]	200 - 12KB - /main.js
[10:30:45]	200 - 3KB - /administration
Task Completed. 2104 requests sent.	

5. DETAILED TECHNICAL FINDINGS & IMPACT ANALYSIS

 FINDING #1: Sensitive Directory Exposure (/ftp)

- **Vulnerability Type:** Directory Listing / Information Leakage
- **Risk Rating:**  **High** (CVSS v3.1 Score: 7.5 | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
- **Technical Description:** Fuzzing activities discovered that the directory /ftp/ is actively exposed and readable by unauthenticated users. The directory directly lists internal, configuration, and backup documents such as .bak and .json files.
- **Impact Analysis:** Unauthenticated users can access proprietary system maps, database templates, and sensitive business documentation (e.g., coupons_2026.pdf.bak, security_audit_stage1.json). An attacker could weaponize this configuration data to bypass business logic or identify other high-value endpoints.

Evidence / Raw Request Proof:

```
GET /ftp/ HTTP/1.1
Host: 192.168.56.102:3000

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

Directory Listing of /ftp/:
- coupons_2026.pdf.bak           [Size: 450 Bytes]
- security_audit_stage1.json    [Size: 1.2 KB]
- package.json.bak              [Size: 840 Bytes]
```

 FINDING #2: Missing Anti-Clickjacking Headers (X-Frame-Options)

- **Vulnerability Type:** Missing HTTP Security Header
- **Risk Rating:**  **Medium** (CVSS v3.1 Score: 4.3 | CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:N/I:L/A:N)
- **Technical Description:** The server response headers do not include either X-Frame-Options or a Frame-Ancestors directive within a Content-Security-Policy. This omission allows the target application's pages to be embedded within <iframe> tags hosted on foreign, malicious sites.

- **Impact Analysis:** Attackers can leverage this misconfiguration to execute Clickjacking attacks. In a Clickjacking scenario, the attacker overlays an invisible iframe of OWASP Juice Shop on top of an attractive, deceptive web page. When the user attempts to click an element on the attacker's page, they unknowingly click hidden actions on Juice Shop, potentially executing unauthorized transactions or changing account security configurations.

Evidence / Raw Request Proof:

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
Content-Length: 12540
ETag: W/"310c-9Gk7Y"
Date: Mon, 20 Jul 2026 10:23:45 GMT
Connection: keep-alive
```

(Anti-Clickjacking / X-Frame-Options header is absent from this block)

● **FINDING #3: Lack of Robust Content-Security-Policy (CSP)**

- **Vulnerability Type:** Missing HTTP Security Header
 - **Risk Rating:** ● **Medium** (CVSS v3.1 Score: 6.1 | CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N)
 - **Technical Description:** The web server does not return a Content-Security-Policy (CSP) header in its web response packets.
 - **Impact Analysis:** CSP is a crucial layer of defense against Cross-Site Scripting (XSS) and data injection vulnerabilities. Without a CSP, if any input fields on the web page are vulnerable to raw script execution, an attacker can easily inject and execute arbitrary JavaScript code within the browsers of visiting users. This could be used to harvest session cookies, hijack client accounts, or deface the interface.
 - **Evidence / Raw Request Proof:** Confirmed by Nikto scanning and manual header inspection via Burp Suite; no CSP directives were identified in standard HTTP payload headers.
-

● **FINDING #4: HTTP Method Enabling (PUT/DELETE Active)**

- **Vulnerability Type:** Weak Web Server Configuration
- **Risk Rating:** ● **Medium** (CVSS v3.1 Score: 5.3 | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:N)
- **Technical Description:** The application allows non-standard and dangerous HTTP verbs (such as PUT and DELETE) on default paths instead of restricting operations purely to standard read-write methods (GET, POST, HEAD).
- **Impact Analysis:** If end-user directories or database endpoints are poorly authorized, an unauthenticated attacker can use the PUT command to upload malicious shells/scripts or use the DELETE command to destroy backend assets. This bypasses the typical web interface restrictions entirely.

Evidence / Raw Request Proof:

```
OPTIONS / HTTP/1.1
Host: 192.168.56.102:3000
```

```
HTTP/1.1 200 OK
Allow: GET, HEAD, POST, PUT, DELETE, OPTIONS
```

● **FINDING #5: Information Disclosure via X-Powered-By Header**

- **Vulnerability Type:** Information Disclosure
- **Risk Rating:** ● **Low** (CVSS v3.1 Score: 3.7 | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
- **Technical Description:** The web application returns an explicit header (X-Powered-By: Express), verifying that it utilizes the Node.js Express framework backend.
- **Impact Analysis:** While information disclosure of this nature does not directly compromise the system, it significantly aids attackers during the reconnaissance phase. Knowing the exact framework and underlying technology helps attackers search for known vulnerabilities (CVEs), tailor their exploitation strategies, and launch highly targeted attacks against the server.

Evidence / Raw Request Proof:

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
```

6. REMEDIATION & ACTION PLAN

To mitigate the discovered vulnerabilities and transition the staging lab into a secure enterprise posture, the following remediation tasks are recommended:

STEP 1: Implement Express Security Headers (Fixes Findings #2, #3, #5)

Configure the Express application or the reverse proxy (such as Nginx) to enforce security headers automatically using middleware libraries like `helmet` in Node.js.

Remediation Code (Node.js/Express):

```
const express = require('express');
const helmet = require('helmet');
const app = express();

// Helmet automatically handles X-Frame-Options, CSP, X-Content-Type-Options, and hides X-Powered-By
app.use(helmet());
```

Manual HTTP Headers to Add (Proxy Config):

```
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline';
```

STEP 2: Restrict Sensitive Directories (Fixes Finding #1)

Explicitly configure routing blocks to disallow access to directory indexes, and remove backup/diagnostic files from public-facing server folders.

- Disable directory indexing on the server.
- Ensure that the `/ftp` folder is placed outside the web server's static routing root, or add authorization middleware checks to restrict its access to authenticated administrators only.

STEP 3: Disable Unsafe HTTP Verbs (Fixes Finding #4)

Limit HTTP methods to only safe verbs required for application usage.

- Configure web routing parameters to drop and reject any HTTP PUT or DELETE requests that do not originate from properly authorized sessions with granular API tokens.

7. CONCLUSION & NEXT STEPS

This vulnerability assessment successfully mapped the sandbox environment's vulnerabilities under safe, simulated conditions. This completes the objectives for **Task-3**.

SUMMARY OF NEXT STEPS

1. **Document Findings:** Save these findings as the foundation of our threat landscape reporting.
2. **Prepare for Exploitation (Task-4):** This report will serve as our guide to target vulnerabilities during the next module, demonstrating the realistic consequences of leaving issues unpatched.
3. **Draft Patches:** Following Task-4, we will apply the remediation code blocks outlined above to completely secure the host system against scanning and exploits.

Report Compiled By: Harshil

Position: Cybersecurity Analyst Intern, Maincrafts Technology